
HYBRID SEARCH INTERNALS

ONNX Runtime | Dense vs Sparse Vectors | Qdrant Storage | FastEmbed | RRF
Fusion

ONNX | DENSE/SPARSE | HNSW | INVERTED INDEX | RRF

- 1 What is ONNX?
- 2 Dense vs Sparse Vectors
- 3 How Qdrant Stores These Vectors
- 4 What FastEmbed Actually Helps With
- 5 Reciprocal Rank Fusion (RRF)
- 6 Master Summary Table
- 7 Decision Guide: Dense vs Sparse vs Hybrid
- 8 Interview Cheat Sheet (Q&A;)
- 9 One-Page Quick Reference

1 What is ONNX?

ONNX (Open Neural Network Exchange) is an open file format + runtime for representing trained ML models in a framework-agnostic way. A model trained in PyTorch/TensorFlow is exported once into a static .onnx computation graph (input tensor -> matmul -> add bias -> activation -> ... -> output tensor). That graph is then executed by ONNX Runtime, a small, highly optimized C++ engine.

PyTorch checkpoint --export--> .onnx graph --run--> ONNX Runtime (C++)

- No PyTorch install required at inference time (saves ~2GB+ of deps incl. CUDA).
- Graph-level optimizations: operator fusion, constant folding, quantization.
- Runs efficiently on CPU; can target GPU/other accelerators via 'execution providers'.
- This is exactly why FastEmbed's SPLADE model (prithivida/Splade_PP_en_v1) loads an ONNX file, not a raw PyTorch checkpoint -- enabling fully offline, no-GPU, no-API-key local inference.

TIP: Analogy: PyTorch is the whole kitchen (recipe book, ingredients, stove, appliances) kept around every time you cook. ONNX freezes the finished dish's exact steps into a compact, portable, ready-to-execute plan -- no kitchen required, just the 'ONNX oven' (runtime) to reheat it.

2 Dense vs Sparse Vectors

The core conceptual split behind any hybrid-search system. Dense vectors capture semantic/conceptual meaning; sparse vectors capture lexical/keyword overlap.

Aspect	Dense Vector	Sparse Vector
Shape	Fixed length (e.g. 768 floats), every position filled	Variable length, ~99% positions are exactly 0
Dimension meaning	Abstract learned axis -- not human-interpretable	Each non-zero index = a specific vocabulary token
Produced by	Vertex AI text-embedding-005	SPLADE (Splade_PP_en_v1) via FastEmbed
Captures	Semantic similarity, synonyms, paraphrases	Exact term matches, with learned expansion
Storage form	[0.021, -0.114, ..., 0.003] (768 floats, always)	indices=[4567,8901,...], values=[2.31,1.87,...]
Similarity metric	Cosine similarity	Dot product over overlapping indices (like BM25)
Strength	Matches meaning even with zero shared words	Exact-match on jargon, acronyms, IDs, column names
Weakness	Can blur distinct-but-related concepts	Can't understand a fully reworded question

Code correspondence:

- Dense path (shared/embedding_service.py): every text -> normalized 768-dim float array (settings.VECTOR_SIZE), used with Distance.COSINE in Qdrant.
- Sparse path (shared/bm25_encoder.py): SPLADE's vocabulary head outputs a ~30,000-dim vector; only non-zero (index, value) pairs are kept as a SparseVector.

KEY IDEA: That's literally why they're named 'dense' (every slot filled) vs 'sparse' (almost all slots empty).

3 How Qdrant Stores These Vectors

Qdrant stores both vector types on the same point (same document), under different named vectors: "dense" and "sparse", declared in the collection schema (vectors_config + sparse_vectors_config).

Internally:

- Dense side -> Qdrant builds an HNSW graph (Hierarchical Navigable Small World): a graph-based approximate nearest-neighbor index. Each point is a node connected to nearby points in 768-D space; queries walk the graph instead of comparing every point.
- Sparse side -> Qdrant builds an inverted index, the same structure real search engines (Elasticsearch/BM25) use. For every non-zero vocabulary index, it keeps a list of which points have a non-zero weight there. Query time = look up only documents sharing at least one token, then compute the dot product.
- on_disk=False on the sparse index keeps the inverted index fully in RAM for speed (vs paging from disk).

```
1 Qdrant point =  
1 ID + 1 dense float[768] + 1 sparse (indices[], values[]) + JSON payload
```

INFO: Both indexes are maintained side-by-side for the same set of points -- this is what lets a single query_points() call search both simultaneously.

4 What FastEmbed Actually Helps With

FastEmbed's one job: be the lightweight bridge that turns raw text into a sparse (or dense) vector without you managing a transformer model yourself.

Without FastEmbed you'd have to manually:

- Download the SPLADE checkpoint from HuggingFace.
- Load it with transformers + PyTorch (heavy dependency, ~GBs).
- Write tokenization -> forward pass -> max-pooling-over-vocabulary logic yourself.
- Manage batching, padding, and thresholding of near-zero values.

FastEmbed abstracts all of that into one call:

```
SparseTextEmbedding(model_name="prithivida/Splade_PP_en_v1")  
encoder.embed(batch) -> clean (indices, values) sparse results
```

TIP: It handles downloading + caching the ONNX model, tokenizing text, running it through ONNX Runtime, and returning results ready for `qm.SparseVector`. FastEmbed offers the same no-PyTorch, ONNX-backed philosophy for dense models too (TextEmbedding, e.g. BAAI/bge, all-MiniLM) -- not used here since Vertex AI covers the dense leg instead.

5 Reciprocal Rank Fusion (RRF)

The problem: two ranked lists (dense results, sparse results) use different, incomparable scoring scales -- cosine similarity (~0 to 1) vs sparse dot-product (any positive value, no fixed range). You can't just average raw scores.

RRF's insight: ignore the raw score entirely; only look at rank position in each list. Ranks are always comparable regardless of the underlying scoring method.

$$\text{RRFscore}(d) = \text{SUM}_{\{i=1..n\}} 1 / (k + \text{rank}_i(d))$$

d = a document; n = number of ranked lists fused ($n=2$: dense + sparse); $\text{rank}_i(d)$ = position of d in list i (1st = 1, 2nd = 2, ...; if d is absent from list i , that term contributes 0); k = smoothing constant, standard default 60 (what Qdrant uses internally).

$$\text{RRFscore}(d) = 1/(60 + \text{rank}_{\text{dense}}(d)) + 1/(60 + \text{rank}_{\text{sparse}}(d))$$

INFO: Why $k=60$? It dampens the impact of very high ranks so rank 1 vs rank 2 doesn't create a huge score gap ($1/61$ vs $1/62$ are close), making fusion robust to noise in any single ranker while still preferring lower ranks overall.

Worked example -- query: "net collection rate"

Document	Dense rank	Sparse rank	RRF score
A (exact term + explains well)	2	1	$1/62 + 1/61 = 0.03253$
B (semantic match, no exact term)	1	not in top-K	$1/61 + 0 = 0.01639$
C (term buried, unrelated context)	not in top-K	3	$0 + 1/63 = 0.01587$

KEY IDEA: Result ranking: $A > B > C$. Document A wins because it scored well on BOTH signals -- that's the entire point of fusion. A doc decently ranked in both lists beats a doc that is #1 in only one list.

In practice, Qdrant computes RRF server-side via `query_points()` with `prefetch=[Prefetch(using="dense",...), Prefetch(using="sparse",...)]` and `query=FusionQuery(fusion=Fusion.RRF)` -- you never implement the formula yourself.

CAUTION: `prefetch_limit = limit x 3` (a common multiplier): RRF needs a WIDER candidate pool from each leg before fusing, because a doc that's mediocre in one leg but excellent in the other must still be PRESENT in both rank lists to get credit. Fetching only top-K per leg risks missing a 'good-in-both' doc that ranked, say, #5 in the dense list.

6 Master Summary Table

Concept	One-line definition	Key formula / mechanism
ONNX	Framework-agnostic model format + runtime for inference	PyTorch checkpoint -> .onnx graph -> ONNX Runtime (C++)
Dense vector	Fixed-length float array capturing semantic meaning	Cosine similarity; every dim filled
Sparse vector	Mostly-zero vector capturing lexical/keyword overlap	Dot product over shared indices; only non-zeros stored
HNSW	Approximate nearest-neighbor graph index for dense vectors	Graph walk to nearby nodes in embedding space
Inverted index	Token -> document-list index for sparse vectors	Same structure as BM25 / Elasticsearch
FastEmbed	Lightweight, PyTorch-free wrapper around ONNX embedding models	download+cache ONNX -> tokenize -> run -> (indices, values)
RRF	Rank-based fusion of multiple ranked result lists	$\text{SUM } 1/(k + \text{rank}_i(d))$, k=60 default

7 Decision Guide: Dense vs Sparse vs Hybrid

Scenario	Best approach	Why
Query rephrased with totally different words ("how well are we collecting?")	Dense	Needs semantic/conceptual matching, no shared keywords
Query has exact jargon/column names ("net_collection_rate")	Sparse (SPLADE/BM25)	Exact-match retrieval on precise terms, IDs, acronyms
Production RAG / search system	Hybrid (dense + sparse + RRF)	Covers both semantic and lexical gaps; best of both signals
Need to run fully offline, no GPU, small footprint	ONNX Runtime via FastEmbed	No PyTorch/CUDA dependency chain; small, fast, portable
Merging 2+ differently-scaled ranked lists	RRF fusion	Rank position is always comparable; raw scores are not

8 Interview Cheat Sheet (Q&A;)

Q. What problem does ONNX solve?

A. It lets a model trained in one framework (PyTorch/TF) run on a small, framework-free C++ runtime, avoiding heavy dependencies (PyTorch+CUDA, ~2GB+) at inference time, while applying graph-level optimizations like operator fusion and quantization.

Q. What is the fundamental difference between dense and sparse vectors?

A. Dense: fixed-length, every dimension filled, dimensions are abstract learned axes of meaning (semantic similarity via cosine). Sparse: variable/very high dimensional, ~99% zeros, each non-zero index maps to an actual vocabulary token (lexical overlap via dot product, like BM25).

Q. Why can't you just average a cosine score and a sparse dot-product score?

A. They live on different, incomparable scales (cosine is bounded ~0-1; dot product is unbounded and depends on term weights), so a raw average is meaningless. You need a scale-free way to combine them, e.g. RRF, which uses rank position instead of raw score.

Q. What is RRF and what is its formula?

A. Reciprocal Rank Fusion: $RRFscore(d) = \text{SUM over lists of } 1/(k + \text{rank}_i(d))$, with $k=60$ typically. It fuses multiple ranked lists by rank position, so a document ranked well in BOTH lists beats one ranked #1 in only one list.

Q. Why is $k=60$ used in RRF?

A. It smooths/dampens the score difference between adjacent ranks (e.g. $1/61$ vs $1/62$ are close), making the fusion robust to noise in any single ranker while still favoring lower (better) ranks overall.

Q. Why fetch `prefetch_limit = 3x` the final limit before fusing with RRF?

A. A document that's excellent in one leg (e.g. sparse) but only mediocre in the other (e.g. rank #5-10 in dense) must still appear in BOTH candidate lists to get fused credit. Fetching a wider pool per leg increases the chance of that overlap before truncating to the final top-K.

Q. What index structure does Qdrant use for dense vs sparse vectors?

A. Dense: HNSW (Hierarchical Navigable Small World), an approximate nearest-neighbor graph. Sparse: an inverted index, the same core structure as BM25/Elasticsearch, mapping each vocabulary token to the documents/weights that contain it.

Q. What does FastEmbed actually do under the hood?

A. It downloads and caches a pre-converted ONNX version of an embedding model (e.g. SPLADE), tokenizes input text, runs it through ONNX Runtime (no PyTorch needed), and returns ready-to-use dense or sparse vectors (for sparse: indices + values).

Q. Why does using an ONNX-based model mean the app can run 'fully offline, no API key'?

A. Because inference happens locally via the small ONNX Runtime engine against a cached .onnx file -- there's no call to an external embedding API and no heavyweight PyTorch/GPU install required, so it's self-contained and works without network access.

Q. In a Qdrant point, what gets stored per document?

A. One ID, one dense float vector (fixed length, e.g. 768), one sparse vector as (indices[], values[]) pairs, and an arbitrary JSON payload (original text, metadata).

9 One-Page Quick Reference

ONNX pipeline:

PyTorch/TF model --export--> .onnx graph --run--> ONNX Runtime (C++, no PyTorch)

Dense vector: fixed-length float[N], cosine similarity, semantic match

Sparse vector: (indices[], values[]) mostly-zero, dot product, lexical match

Qdrant index: dense -> HNSW graph | sparse -> inverted index (BM25-style)

RRF formula:

$\text{RRFscore}(d) = \text{SUM}_{\{i=1..n\}} 1 / (k + \text{rank}_i(d))$ [k = 60 default]

2-leg case: $1/(60 + \text{rank_dense}(d)) + 1/(60 + \text{rank_sparse}(d))$

prefetch_limit = limit x 3 (wider pool per leg so RRF can find good-in-both docs)

KEY IDEA: Golden rule: dense = meaning, sparse = exact words, RRF = fair way to combine ranked lists that use incomparable score scales.